

iSAQB Module: Formal Methods - VeriFast

Concepts and Syntax

Active Group GmbH

Created: 2026-10-04 Sun 12:49

Basic VeriFast Syntax

- VeriFast specifications are embedded in C comments

- single-line annotation:

```
| //@ ...
```

- multi-line annotations:

```
| /*@  
|   ...  
| @*/
```

- annotations are ignored by the C compiler, but interpreted by VeriFast
- separation conjunction is written $\&*\&$

```
| P &*& Q
```

- P and Q hold for **separate** resources

The Symbolic State

- VeriFast **symbolically executes** the program
- at each execution step, its current symbolic state contains:
 - **Locals**: values of local variables
 - **Assumptions**: facts known to hold
 - **Heap**: currently available heap chunks
- statements and annotations transform the symbolic state as execution proceeds

Assumptions

- Assumptions are facts VeriFast knows to hold on the current execution path
- they arise from, e.g.:
 - function preconditions
 - conditionals
 - assignments and other program statements

- example:

```
if (x < y) {  
    // here VeriFast assumes x < y  
    ...  
} else {  
    // here VeriFast assumes !(x < y)  
    ...  
}
```

- VeriFast uses these facts to prove subsequent conditions

The Symbolic Heap

- the symbolic heap tracks the memory resources currently available to VeriFast
- these resources are represented by **heap chunks**
- memory can only be accessed if the required heap chunk is available

→ more on heap chunks shortly

Function Contracts

- every function is specified by a **contract**
 - **requires** specifies the **precondition**
 - **ensures** specifies the **postcondition**

```
int abs(int x)
    //@ requires true;
    //@ ensures result >= 0;
{
    return x < 0 ? -x : x;
}
```

- the simplest possible contract is:

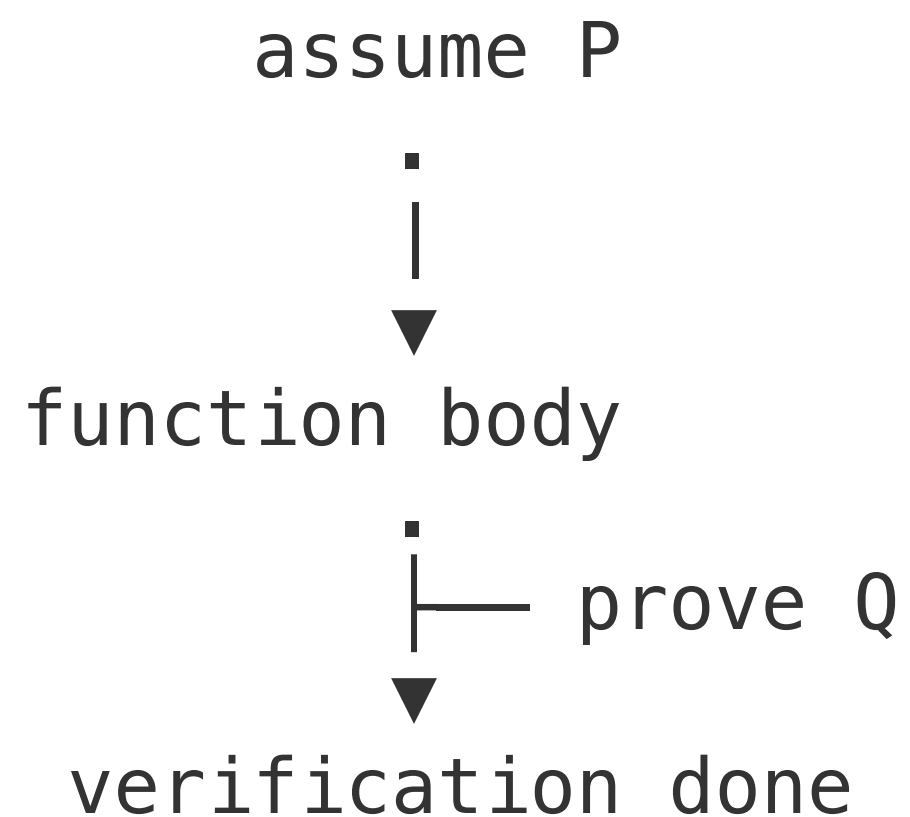
```
//@ requires true;
//@ ensures true;
```

Verifying a Function

Given a function with

```
//@ requires P;  
//@ ensures Q;
```

- VeriFast **assumes** the precondition P
- it symbolically executes the function body
- every possible execution path must establish the postcondition Q



Calling a Function

Given a call to a function with

```
//@ requires P;  
//@ ensures Q;
```

- the function body is **not** evaluated at the call site
- VeriFast checks that P follows from the current symbolic state
- P is **consumed**
- Q is added to the symbolic state

current symbolic state

┆
┆ — prove P



function call

┆
┆ — obtain Q



new symbolic state

→ verification is **modular**: callers reason about a function through its contract, not its implementation

Heap Chunks

- a heap chunk represents
 - **ownership** of a memory location
 - information about the value stored there
- generic syntax for struct fields:
 - | `State_x(s, x)`
 - `State_x` identifies the `x` field of a `State` struct
 - `s` identifies the struct instance
 - `x` is the value stored in the field
- having the chunk gives us permission to access the field

Initialized and Uninitialized Memory

- heap chunks distinguish initialized from uninitialized memory

```
| State_x_(s, dummy)    // uninitialized  
| State_x(s, 42)        // initialized
```

- after `malloc`, the fields exist but are **uninitialized**
- assigning a value initializes the corresponding field

```
| State *s = malloc(sizeof(State));  
| // State_x_(s, dummy)  
  
| s->x = 0;  
| // State_x(s, 0)
```

- uninitialized memory may be **written**, but not **read**
- additionally, `malloc` produces a chunk for the allocated block:

```
| malloc_block_State(s)
```

- this chunk is only required to eventually `free(s)`

Struct Field Chunks

- VeriFast provides a more convenient syntax for struct fields

State_x_(s, _)	≡	s->x	->	_
State_x(s, x)	≡	s->x	->	x

- s->x |-> _
 - ownership of an **uninitialized** field
- s->x |-> x
 - ownership of an **initialized** field
 - the field contains x
- prefer the field syntax in specifications

Referring to Values

- `result` refers to the **return value** of a function

```
int abs(int x)  
  //@ requires true;  
  //@ ensures result >= 0;
```

- `?x` introduces a **pattern variable**
 - binds an otherwise unknown value to the name `x`
 - the name can subsequently be used in the contract

```
//@ requires p->value |-> ?x;  
//@ ensures p->value |-> x &* & result == x;
```

- `_` is a **wildcard**
 - matches a value without giving it a name
 - useful when the concrete value is irrelevant

```
//@ requires p->value |-> _;
```

Predicates

- predicates **package assertions** under a common name

```
/*@  
predicate point(Point *p, int x, int y) =  
    p->x |-> x &*&  
    p->y |-> y &*&  
    0 <= x &*& 0 <= y;  
@*/
```

- predicates can combine
 - heap chunks
 - assumptions about their values
- a predicate can be used like any other assertion

```
//@ requires point(p, x, y);
```

Closing Predicates

- VeriFast does not automatically create predicate instances
- if the assertions of a predicate are available, we can **close** it

```
| //@ close point(p, x, y);
```

$p \rightarrow x \mid \rightarrow x$

$p \rightarrow y \mid \rightarrow y$

$0 \leq x$

$0 \leq y$

•
| close
▼

$\text{point}(p, x, y)$

- closing **consumes** the assertions and produces the predicate instance

Opening Predicates

- assertions packaged inside a predicate are not directly available
- to access them, we **open** the predicate

```
| //@ open point(p, x, y);
```

```
point(p, x, y)
```

↓ open

```
p->x |-> x  
p->y |-> y  
0 <= x  
0 <= y
```

- opening **consumes** the predicate instance and produces its assertions

Assertions

- `assert` asks VeriFast to prove that an assertion follows from the current symbolic state

```
| //@ assert x < y;
```

- if VeriFast can prove the assertion, verification continues
- otherwise, verification fails at that point

- assertions can also expose values from heap chunks

```
| //@ assert p->x |-> ?x;
```

- after the assertion, `x` refers to the value stored in `p->x`
- assertions are useful for making intermediate reasoning steps explicit and for understanding why a proof fails

Loop Invariants

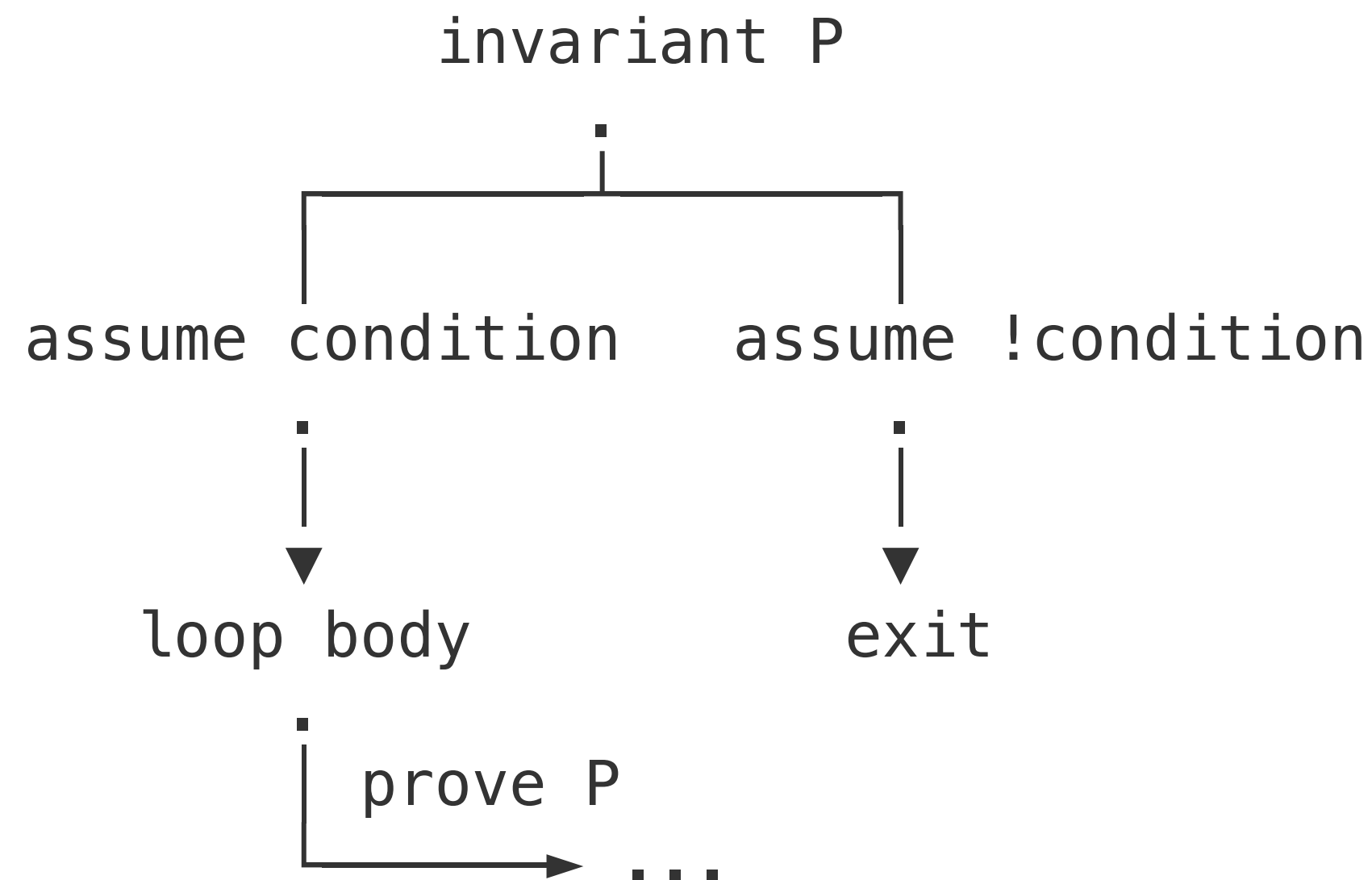
- a loop invariant describes what must hold **between iterations**

```
while (condition)
    //@ invariant P;
{
    ...
}
```

- VeriFast verifies the loop by checking:
 - P holds when the loop is first reached
 - assuming P and condition, one iteration re-establishes P
 - after the loop, P and !condition hold

Verifying a Loop

- at the loop head, VeriFast keeps only what is described by the invariant
- for an iteration, it assumes:
 - the loop invariant
 - the loop condition
- after the loop, it knows:
 - the loop invariant
 - the negation of the loop condition



Finding the Right Loop Invariant

- an invariant must contain enough information to verify:
 - the loop body
 - the invariant itself for the next iteration
 - what should hold after the loop
- finding such an invariant is usually **incremental**
- start with what you expect to be preserved by the loop
- if verification fails, inspect the symbolic state:
 - what fact or heap chunk is missing?
 - was it available before entering the loop?
 - does it need to become part of the invariant?
- strengthen the invariant and try again

Termination

- a loop invariant establishes **partial correctness**
 - if the loop terminates, the desired properties hold
- to prove that the loop **terminates**, we provide a **measure** using a decreases clause

```
for (int i = 0; i < n; i++)  
    //@ invariant 0 <= i &*& i <= n;  
    //@ decreases n - i;  
{  
    ...  
}
```

- the measure must
 - be **non-negative**
 - **strictly decrease** with every iteration
- therefore, the loop cannot execute indefinitely

Going Further

Ghost State

- sometimes verification needs information that the program itself does not need to store
- ghost state exists only for verification
 - it does not affect program execution
 - it is erased when compiling the C program

```
//@ int old_x = x;  
  
x++;  
  
//@ assert x == old_x + 1;
```

- ghost variables can be updated just like ordinary variables
- use ghost state to remember information needed for reasoning, without changing the program

Fixpoint Functions

- fixpoint functions define pure mathematical functions for use in specifications

```
/*@  
fixpoint int square(int x) {  
    return x * x;  
}  
@*/
```

- they can be used in contracts, predicates, assertions, ...

```
//@ ensures result == square(x);
```

- fixpoint functions
 - have no side effects
 - operate only on verification-level values
 - are not executed as part of the C program

Lemmas

- sometimes a required fact does not follow automatically from VeriFast's built-in reasoning
- a lemma is a ghost function used to prove such a fact

```
/*@  
lemma void example_lemma(int x)  
    requires 0 <= x;  
    ensures 0 <= x * x;  
{  
    ...  
}  
@*/
```

- calling the lemma makes its postcondition available

```
//@ example_lemma(x);
```

- lemmas are used only during verification and do not become part of the executable program

Putting It Together: The Verification Workflow

1. write a contract describing what the function requires and guarantees
2. let VeriFast symbolically execute the program
3. when verification fails, inspect the symbolic state
 - which assertion could not be established?
 - which fact or heap chunk is missing?
4. add the specification or intermediate reasoning that is actually needed
 - predicates
 - assertions
 - loop invariants
 - ghost state
 - ...
5. verify again

→ verification is an **iterative process**: specify, verify, inspect, refine
and most importantly: **Have fun! :)**